GIT TEAM COLLABORATION

Complete Beginner Guide — Two Workflows

Fork Workflow | Same Repository Workflow

Fork Workflow	&	Same Repo Workflow
Open source contribution style. You copy the entire repo to your own GitHub account, work there, then send a Pull Request back to the original.		Company / team project style. Everyone has direct access to one shared repository. Create a branch, do your work, and open a Pull Request.

Step-by-step · Every command explained · Real examples with GitHub

TABLE OF CONTENTS

PART 1

Understanding Team Collaboration in Git

What is Collaboration in Git?

When you work alone, Git is just version control. When you work in a TEAM, Git becomes a powerful collaboration system. The key idea is: **everyone works on their own copy or branch, then brings the changes together.**

No one works directly on the main branch. Everyone creates a separate workspace, does their work, and then requests that it be added to the main codebase. This way, no one accidentally breaks the project for everyone else.

The Two Main Collaboration Workflows

Workflow	Best For
Fork Workflow	Open source projects. You do NOT have write access to the original repo. You make your own copy (fork) and contribute through Pull Requests.
Same Repo Workflow	Company or private team projects. Everyone has direct access (write permission) to the same repository. You create branches on the shared repo.

Key Concepts You Must Know

Term	What It Means
Fork	Your personal copy of someone else's repository, stored on YOUR GitHub account. Changes in your fork do NOT affect the original repo.
Clone	Downloading a repository from GitHub to your local computer. After cloning you can work offline.
Remote	A shortcut name pointing to a URL on GitHub. "origin" usually means YOUR fork or repo. "upstream" means the original repo.
Upstream	The original repository that your fork was made from. You pull new changes FROM upstream.
Origin	The remote pointing to your own fork or repo on GitHub. You push TO origin.
Pull Request (PR)	A request on GitHub to merge your branch/fork changes into the main codebase. This is where code review happens.
Branch	An isolated line of development. Create one for each feature or bug fix so it does not affect main code.

PART 2

FORK WORKFLOW — Step by Step

What is a Fork?

A fork is like making a photocopy of someone's entire notebook. The original stays with them. You get your own copy to write in freely. When you want your improvements added back to the original, you send a request.

```
# Visual: The Fork Relationship
company/project (ORIGINAL — you cannot push here directly)
|
| click Fork on GitHub
|
yourname/project (YOUR FORK — you own this, push freely)
|
| git clone
|
/your-laptop/project (LOCAL — where you write code)
```

■■ NOTE:

USE THIS WHEN: You are contributing to open source projects (like Django, React, etc.) where you do NOT have write access to the original repository.

Step 1 — Fork the Repository on GitHub

1

Fork on GitHub (No commands needed — done on the website)

1. Go to the original repository on GitHub (e.g. github.com/company/project)
2. Click the **"Fork"** button at the top-right of the page
3. Select your account as the destination
4. GitHub creates a copy at: **github.com/yourname/project**

Result:

```
# Before fork:
github.com/company/project ← original (read-only for you)
# After fork:
github.com/yourname/project ← YOUR copy (you are the owner!)
github.com/company/project ← original still exists, unchanged
```

Step 2 — Clone YOUR Fork to Your Computer

Now you download your fork (NOT the original) to your laptop. This is where you will write code.

2 Clone your fork locally

Clone from YOUR fork URL (yourname/project), not the original.

Command to run in your Terminal

```
# Clone YOUR fork (replace "yourname" and "project" with real values)
git clone https://github.com/yourname/project.git
# Go into the project folder
cd project
# Verify: check what remote was set up
git remote -v
# Output:
# origin https://github.com/yourname/project.git (fetch)
# origin https://github.com/yourname/project.git (push)
# "origin" = YOUR fork on GitHub (good!)
```

■ TIP:

At this point "origin" points to YOUR fork. Any git push will go to your fork, NOT to the original company/project. That is exactly what we want!

Step 3 — Add the Original Repo as "upstream"

This is the most important step beginners miss! You need to connect to the **original repository** so you can get new changes from it. We give it the nickname "upstream".

3 Add upstream remote — connect to the original repo

This lets you fetch new updates from the original project.

Connect to the original repository

```
# Add the ORIGINAL repo as "upstream"
git remote add upstream https://github.com/company/project.git
# Now verify — you should see BOTH remotes:
git remote -v
# Output (what you want to see):
# origin https://github.com/yourname/project.git (fetch) ← YOUR fork
# origin https://github.com/yourname/project.git (push) ← YOUR fork
# upstream https://github.com/company/project.git (fetch) ← ORIGINAL
# upstream https://github.com/company/project.git (push) ← ORIGINAL
```

Remote Name	Points To
origin	YOUR fork on GitHub — you push here
upstream	The ORIGINAL repo — you only fetch/pull from here

Step 4 — Create a Feature Branch

NEVER work directly on main. Always create a new branch for each feature or bug fix. This keeps your work isolated and makes Pull Requests clean.

4 **Create and switch to a new branch**

Name your branch descriptively so everyone knows what it does.

Creating your working branch

```
# Create a new branch and switch to it
git checkout -b feature-login
# Or using modern syntax:
git switch -c feature-login
# Good branch naming examples:
# feature-login
# fix-grade-bug
# feature/user-registration
# fix/null-pointer-error
# Verify you are on the new branch:
git branch
# Output:
# main
# * feature-login ← asterisk shows current branch
```

Step 5 — Make Your Changes and Commit

Now write your code! Create files, edit files, do your work. Then stage and commit your changes with clear messages.

5 **Edit files, stage, and commit**

Work freely — your changes are only on your local branch.

Make changes and commit

```
# Example: create a new file
echo "def login(user, password): pass" > login.py
# Check what changed
git status
# Stage your changes
git add login.py
# Or stage everything: git add .
# Commit with a clear message
git commit -m "feat: add login function"
# You can make multiple commits — each one is a saved checkpoint
```

```
echo "def logout(user): pass" >> login.py
git add login.py
git commit -m "feat: add logout function"
# See your commits
git log --oneline
```

Step 6 — Push to YOUR Fork

Upload your local branch to YOUR fork on GitHub. Remember: this goes to your fork (origin), NOT to the original repo.

6 **Push your branch to GitHub (your fork)**

This makes your branch visible on GitHub so you can create a PR.

Push to your fork on GitHub

```
# Push your branch to YOUR fork (origin)
git push origin feature-login
# Output:
# Enumerating objects: 3, done.
# Counting objects: 100% (3/3), done.
# To https://github.com/yourname/project.git
# * [new branch] feature-login -> feature-login
# After this, go to GitHub.com and open YOUR fork.
# You will see a yellow banner:
# "feature-login had recent pushes — Compare & pull request"
```

■■ NOTE:

The push goes to origin (your fork). The original company/project repo is NOT affected yet. Only after your Pull Request is reviewed and merged will your code appear there.

Step 7 — Create a Pull Request (PR)

A Pull Request is your formal request saying: "I have made changes in my fork — please review and add them to the original project." This is done entirely on the GitHub website.

7 **Open a Pull Request on GitHub**

All done on the GitHub website — no terminal commands needed here.

Creating the Pull Request on GitHub

```
# STEPS ON GITHUB WEBSITE:
1. Go to YOUR fork: github.com/yourname/project
2. Click the green "Compare & pull request" button
(or go to Pull Requests tab -> New Pull Request)
3. Check the settings:
```

```
base repository: company/project ← where you want to merge INTO
base branch: main ← which branch on original
head repository: yourname/project ← your fork
compare branch: feature-login ← your branch
4. Write a good PR description:
Title: "feat: add login and logout functions"
Description:
- What does this PR do?
- Why was this change needed?
- How to test it?
5. Click "Create Pull Request"
# The maintainer will now see your PR and review it.
```

Step 8 — Maintainer Reviews and Merges

After you submit your PR, the project maintainer reviews your code. They may ask for changes, approve it, or reject it.

What happens after creating a PR

```
# What happens after you submit a PR:
# SCENARIO A: Maintainer requests changes
# - They leave comments like "Please add error handling here"
# - You make the changes on your LOCAL branch:
echo "def login(user, password):" > login.py
echo " if not user: raise ValueError('user required')" >> login.py
git add login.py
git commit -m "fix: add error handling to login"
git push origin feature-login
# - The new commit automatically appears in the SAME PR!
# SCENARIO B: PR is approved
# - Maintainer clicks "Merge pull request"
# - Your code is now in the original repo!
# - You can delete your feature branch (GitHub shows a button for this)
# SCENARIO C: PR is rejected
# - Maintainer explains why
# - You learn, improve, and try again
```

Keeping Your Fork Updated

The original project keeps evolving. Other people's PRs get merged. You must regularly update your fork with those new changes before starting new work. This prevents conflicts.

Syncing your fork with the original repo

```
# DO THIS before starting ANY new feature:
# Step 1: Switch to your main branch
git checkout main
```

```
# Step 2: Fetch all new changes from the original repo
git fetch upstream
# Step 3: Merge those changes into your local main
git merge upstream/main
# Step 4: Push the updated main to YOUR fork on GitHub
git push origin main
# Now your fork is up to date with the original!
# Start your new feature branch from this updated main:
git checkout -b feature/new-thing
```

■■ WARNING:

Always sync your fork BEFORE creating a new branch. If you forget, your branch will be behind and you will get merge conflicts later.

Full Fork Workflow — At a Glance

Complete Fork Workflow

```
# ■■■ ONE-TIME SETUP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 1. Fork on GitHub (click Fork button)
git clone https://github.com/yourname/project.git
cd project
git remote add upstream https://github.com/company/project.git
# ■■■ BEFORE EACH FEATURE (sync fork) ■■■■■■■■■■■■■■■■■■■■■■
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
# ■■■ FOR EACH FEATURE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git checkout -b feature-name
# ... make your changes ...
git add .
git commit -m "feat: describe your change"
git push origin feature-name
# → Open GitHub → Create Pull Request
# ■■■ AFTER PR IS MERGED ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git checkout main
git fetch upstream && git merge upstream/main
git branch -d feature-name # delete local branch
```

PART 3

SAME REPOSITORY WORKFLOW — Step by Step

What is the Same Repository Workflow?

In a company or private team, all developers are added as **collaborators** to ONE shared repository. Everyone has permission to push branches directly. Instead of forking, you create a branch on the shared repo, do your work, and open a PR.

```
# Visual: Same Repository
github.com/company/project (ONE shared repo — everyone can push branches)
| | |
Ravi's Priya's Arjun's
branch branch branch
All branches live in the SAME repository.
No forking needed.
```

■■ NOTE:

USE THIS WHEN: You are working on a private company project or a team repo where everyone has been given "collaborator" access by the repo owner.

Step 1 — Clone the Shared Repository

Everyone on the team clones the SAME repository. You all start from the same place.

1 **Clone the shared company/team repo**

Everyone does this once to set up their local copy.

One-time setup

```
# Clone the team repository (given to you by the repo owner/manager)
git clone https://github.com/company/project.git
# Go into the project
cd project
# Check the remote — you will see just "origin"
git remote -v
# Output:
# origin https://github.com/company/project.git (fetch)
# origin https://github.com/company/project.git (push)
# No need to add "upstream" — origin IS the shared repo!
# First, get the latest code
git pull origin main
```

Step 2 — Create Your Feature Branch

Never work on main directly. Create your own branch. Use a name that identifies you and/or the feature, so others know whose branch it is.

2 Create your branch from the latest main

Always start from the latest main to avoid conflicts.

Creating your feature branch

```
# Make sure you are on main and it is up to date
git checkout main
git pull origin main
# Create a new branch for your feature
git checkout -b feature/ravi-student-search
# Good naming conventions for team branches:
# feature/your-name-what-you-are-doing
# fix/your-name-bug-description
# Examples:
# feature/ravi-login-page
# fix/priya-grade-calculation-bug
# feature/arjun-export-csv
# Verify you are on your new branch:
git branch
# * feature/ravi-student-search ← asterisk = current branch
# main
```

Step 3 — Make Changes and Commit

Write your code. Make as many commits as you need. Each commit should be a logical unit of work with a clear message.

3 Edit, stage, and commit your changes

Write meaningful commit messages — your teammates will read them.

Making and committing changes

```
# Create/edit your files
echo "def search_student(name, students):" > search.py
echo "    return [s for s in students if s['name'] == name]" >> search.py
# Check what changed
git status
# Output: Untracked files: search.py
# Stage the file
git add search.py
# Commit with a descriptive message
git commit -m "feat: add student search by name function"
```

```
# Make another change and commit
echo "def search_by_roll(roll, students):" >> search.py
echo "    return [s for s in students if s['roll'] == roll]" >> search.py
git add search.py
git commit -m "feat: add search by roll number"
# See your commits
git log --oneline
# a3f2c1b feat: add search by roll number
# 9e1d4a2 feat: add student search by name function
```

Step 4 — Push Your Branch to GitHub

Upload your branch to the shared GitHub repository. This makes it visible to all your teammates.

4 **Push your branch to the shared repo**

Unlike fork workflow, this pushes directly to the team repo (not a personal fork).

Push branch to shared repository

```
# Push your branch to origin (the shared team repo)
git push origin feature/ravi-student-search
# Output:
# Enumerating objects: 4, done.
# To https://github.com/company/project.git
#  * [new branch] feature/ravi-student-search -> feature/ravi-student-search
# Now on GitHub, EVERYONE on the team can see your branch!
# After pushing, GitHub shows a banner:
# "feature/ravi-student-search had recent pushes"
# → Click "Compare & pull request" to start the PR
```

Step 5 — Create a Pull Request

Go to GitHub and create a Pull Request. This is the same as the fork workflow, but simpler — everything is in the same repo.

5 **Open a Pull Request on GitHub**

Request to merge your branch into main.

Creating the Pull Request

```
# STEPS ON GITHUB WEBSITE:
1. Go to github.com/company/project
2. Click the yellow banner "Compare & pull request"
   OR go to Pull Requests tab → New Pull Request
3. Settings:
```

```
base branch: main ← merge INTO main
compare branch: feature/ravi-student-search ← YOUR branch
4. Write your PR description:
Title: "feat: add student search functionality"
Description:
## What does this PR do?
Adds two new functions to search students by name and roll number.
## How to test
1. Run python search.py
2. Call search_student("Ravi", students)
5. Assign reviewers (your teammates)
6. Click "Create Pull Request"
```

Step 6 — Code Review and Merge

Your teammates review your code, leave comments, approve, and merge. Here is what each person does:

The review and merge process

```
# ■■ AS THE PR AUTHOR (you) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# If reviewers request changes:
# Make the changes locally on the SAME branch
echo " # handles empty list" >> search.py
git add search.py
git commit -m "fix: handle empty students list in search"
git push origin feature/ravi-student-search
# New commit auto-appears in the PR!
# ■■ AS A REVIEWER (teammate) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# On GitHub:
# → Go to the PR
# → Click "Files changed" to see what changed
# → Click "+" next to any line to leave a comment
# → Click "Review changes" → "Approve" or "Request changes"
# ■■ MERGING THE PR (team lead or author after approval) ■■
# On GitHub:
# → Click "Merge pull request"
# → Click "Confirm merge"
# → Click "Delete branch" (to keep repo clean)
# ■■ AFTER MERGE: update your local main ■■■■■■■■■■■■■■■■■■■■
git checkout main
git pull origin main
git branch -d feature/ravi-student-search # delete local branch
```

Handling Merge Conflicts in Team Setting

Merge conflicts happen when two people edit the same lines of the same file. Here is how to handle it in the same repo workflow:

Resolving merge conflicts

```
# SCENARIO: You and Priya both edited config.py on different branches.
# When your PR is being merged, GitHub shows a conflict.
# STEP 1: Update your branch with latest main
git checkout feature/ravi-student-search
git fetch origin
git merge origin/main
# Output: CONFLICT (content): Merge conflict in config.py
# STEP 2: Open config.py and look for conflict markers:
<<<<<<< HEAD (your version)
DATABASE = "students.db"
=======
DATABASE = "school.db"
>>>>>>> origin/main (main version)
# STEP 3: Edit the file - remove markers, keep correct version:
DATABASE = "school.db" # decided to keep main's version
# STEP 4: Stage and commit the resolution
git add config.py
git commit -m "fix: resolve merge conflict in config.py"
# STEP 5: Push the resolved branch
git push origin feature/ravi-student-search
# Now GitHub PR shows: "This branch has no conflicts"
```

■ TIP:

VS Code makes conflict resolution easy — it shows "Accept Current Change", "Accept Incoming Change", or "Accept Both Changes" buttons. Use them!

Full Same Repo Workflow — At a Glance

Complete Same Repo Workflow

```
# ■■■ ONE-TIME SETUP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git clone https://github.com/company/project.git
cd project
# ■■■ BEFORE EACH FEATURE (get latest) ■■■■■■■■■■■■■■■■■■■■
git checkout main
git pull origin main
# ■■■ FOR EACH FEATURE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git checkout -b feature/yourname-featurename
# ... write your code ...
git add .
git commit -m "feat: describe your change"
git push origin feature/yourname-featurename
# → Open GitHub → Create Pull Request → Assign reviewers
# ■■■ IF CONFLICTS ARISE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git fetch origin
```

```
git merge origin/main
# → resolve conflicts → git add → git commit → git push
# ■■■ AFTER YOUR PR IS MERGED ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
git checkout main
git pull origin main
git branch -d feature/yourname-featurename
```

PART 4

Side-by-Side Comparison

Fork vs Same Repo — Key Differences

Aspect	Fork Workflow	Same Repo Workflow
Who uses it?	Open source contributors, external developers	Company teams, private projects, school projects
Access level?	No write access to original repo	Write access (collaborator) on shared repo
Where do you push?	To YOUR fork (origin = your copy)	To the shared repo (origin = team repo)
Need "upstream"?	YES — to sync with original	NO — origin is already the team repo
One-time setup	Fork → Clone → Add upstream	Clone only
Before new feature	Sync fork: fetch upstream + merge + push	Just: git pull origin main
PR direction	yourname/project → company/project	feature-branch → main (same repo)
Complexity	More steps (2 remotes to manage)	Simpler (1 remote)

When to Use Which Workflow

Your Situation	Use This Workflow
Contributing to Django, React, or any open source project	**Fork Workflow**
Working at a company with your dev team	**Same Repo Workflow**
Classroom / workshop project where teacher shared the repo	**Same Repo Workflow**
You want to suggest improvements to a public repo you don't own	**Fork Workflow**
Private team project (startup, internship, college club)	**Same Repo Workflow**
You are a maintainer receiving contributions from strangers	**Fork Workflow (they fork, you review)**

Common Mistakes and How to Avoid Them

Mistake	How to Avoid
Working directly on main	ALWAYS create a new branch before starting work. Rule: main is read-only for you.
Forgetting to pull/sync before starting new work	Always run git pull origin main (same repo) or sync fork before creating a branch.

Pushing to the wrong remote	In fork workflow: always push to origin (your fork), never to upstream.
Not writing clear commit messages	Use format: type: short description. E.g. "feat: add login function"
Making huge PRs with 50 files changed	Keep PRs small and focused. One feature = one PR. Easier to review.
Force pushing to shared branches	NEVER use git push --force on main or any shared branch. It can destroy everyone's work.
Not deleting merged branches	After PR is merged, delete the branch on GitHub and locally. Keep the repo clean.

QUICK REFERENCE CHEAT SHEET

FORK WORKFLOW

```
# ONE-TIME SETUP
# (1) Fork button on GitHub
git clone https://github.com/YOUR_USERNAME/REPO.git
cd REPO
git remote add upstream https://github.com/ORIGINAL_OWNER/REPO.git
# SYNC FORK (do before each new feature)
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
# FEATURE WORK
git checkout -b feature-name
# edit files
git add .
git commit -m "feat: describe change"
git push origin feature-name
# → GitHub: Create Pull Request
# CLEANUP (after PR merged)
git checkout main
git fetch upstream && git merge upstream/main
git branch -d feature-name
```

SAME REPOSITORY WORKFLOW

```
# ONE-TIME SETUP
git clone https://github.com/COMPANY/REPO.git
cd REPO
# BEFORE EACH FEATURE (get latest)
git checkout main
git pull origin main
# FEATURE WORK
git checkout -b feature/yourname-feature
# edit files
git add .
git commit -m "feat: describe change"
git push origin feature/yourname-feature
# → GitHub: Create Pull Request → Assign reviewers
# IF CONFLICTS
git fetch origin
git merge origin/main
# resolve conflicts in files
git add . && git commit -m "fix: resolve conflicts"
git push origin feature/yourname-feature
```

```
# CLEANUP (after PR merged)
git checkout main
git pull origin main
git branch -d feature/yourname-feature
```

Key Rules to Never Forget

1. NEVER work directly on main. Always create a branch.
2. ALWAYS pull/sync before creating a new branch.
3. Write CLEAR commit messages: type: short description
4. Keep Pull Requests SMALL and focused (one feature per PR).
5. NEVER git push --force on main or shared branches.
6. DELETE your branch after it is merged. Keep things clean.
7. In FORK workflow: push to origin (your fork), fetch from upstream (original).
8. In SAME REPO workflow: origin is the shared team repo — use it for both.
